

Curtin University

Dubai

COMP2002
Unix System Programming

ASSIGNMENT REPORT

FINAL ASSIGNMENT

Prepared By :

Resham Furqan

22526025

Instructor:

Mr. Girish Balasubramanian

Table of Contents

Introduction 2
Reading and Parsing Integers from the Text File 2
Creating and Organizing the Pipes 3
Preventing Blocking Reads in Child Processes 3
Tracking Child Processes and Reading Results 4
Testing and Validation 5
Conclusion 7

Introduction

This program reads integers from a text file (numbers.txt), creates a child process for each integer using fork(), and uses pipes for Inter Process Communication. Each child determines whether its assigned number is prime and sends the result back to the parent for output. Only system calls (open(), read(), write(), close(), fork(), and pipe()) are used throughout the program

Reading and Parsing Integers from the Text File

The program uses only low-level system calls (open(), read(), and close()) to read the numbers from the text file without using “fscanf()” or any other prohibited library function. This is implemented in “fileio.c” inside the function “read_numbers_from_file()”. The function opens the file using:

```c
int* read_numbers_from_file(const char* filename, int* count)
{
    int fd = open(filename, O_RDONLY);
    if (fd < 0)
    {
        const char* msg = "Error: cannot open file\n";
        write(STDERR_FILENO, msg, sizeof("Error: cannot open file\n") - 1);
        _exit(1);
    }
```

Then it reads the entire file content character by character using a while loop:

```c
while (read(fd, &ch, 1) == 1)
{
    if (ch >= '0' && ch <= '9')
    {
        num = num * 10 + (ch - '0'); // converting char digit to int
        has_number = 1;
    } else if (ch == '\n' || ch == ' ' || ch == '\t')
    {
        if (has_number)
        {
            if (is_first)
            {
                total = num;
                is_first = 0;
                *count = total;
                free(numbers);
                numbers = (int*) malloc(total * sizeof(int));
                num_count = 0;
            }else {
                if (num_count < total)
                {
                    numbers[num_count++] = num;
                }
            }
            num = 0;
            has_number = 0;
        }
    }
}
```

It loops through each character in the buffer to manually parse digits and convert them into integers. When it encourages a space or newline, it stores the parsed number into a 1D integer array. This ensures that the integers are correctly separated and stored even if the file contains mixed whitespace characters. At the end, the pointer to this dynamically allocated array and the total count are returned to the main program.

Creating and Organizing the Pipes

The program creates two pipes per child process, one for communication from parent to child, and one for from child back to parent. This is done in “prime.c” using two arrays of pipes:

```
}
int parent_to_child[count][2];
int child_to_parent[count][2];
```

Each call to “pipe()” initializes a pair of file descriptors. For every child created with “fork()”, the parent keeps the write-end of “parent_to_child” and the read-end of “child_to_parent”, while the child keeps the opposite ends. As each child has its own index, both parent and child know which pipe to use, the one corresponding to their own index in the loop.

Preventing Blocking Reads in Child Processes

Each child process performs a single “read()” from its respective “parent_to_child[ii][0]” pipe in the “prime.c” file. The parent carefully manages pipe ends:

```
close(parent_to_child[ii][0]); // parent does not read from this
close(child_to_parent[ii][1]); // parent does not write to this
```

After the parent writes the integer to the pipe, it closes its write-end:

```
// sending the data to children
int j;
for (j = 0; j < count; j++)
{
    write(parent_to_child[j][1], &numbers[j], sizeof(int));
    close(parent_to_child[j][1]); // done writing
}
```

In the child process:

```
close(parent_to_child[ii][0]); // parent does not read from this
close(child_to_parent[ii][1]); // parent does not write to this
```

This is important because once the parent closes the write-end, the child's "read()" will return 0 (EOF) when there is no more data, instead of blocking forever. Therefore, the child knows there are no more numbers to read when read() finishes or returns zero, and then the child exits safely using "_exit(0)"

Tracking Child Processes and Reading Results

The parent keeps an array "pids[count]" in the "prime.c" file to record the process ID of each forked child.

```
pid_t pids[count];
```
and
```
pids[ii] = pid;
```

After sending all numbers, the parent waits for each child using "wait()". Since children may complete in any order, a PID matching algorithm is used:

```
// wait for children and read results
int jj;
for (jj = 0; jj < count; jj++)
{
    int status;
    pid_t cpid = wait(&status);
```

Once a child terminates, the parent matches its PID with the stored one in "pids[count]". When a match is found, it knows which pipe corresponds to that child and reads the result of primality test from:

```
int k;
for (k = 0; k < count; k++)
{
    if (pids[k] == cpid)
    {
        int result;
        read(child_to_parent[k][0], &result, sizeof(int));
```

This ensures that each child's result is read from the correct pipe and associated with the correct number.

Testing and Validation

To verify the correctness and robustness of the program, several test cases were performed:

1. Functional Tests:

numbers.txt, numbers1.txt, small_numbers.txt, single_numbers.txt, and large_numbers.txt were used to confirm that the integers are read correctly from the file using “read()” and stored into the array. Each child process received exactly one integer and returned the correct primality result. The process correctly printed results in the expected format as shown in the following screenshots:

```
resham@resham-VirtualBox:~/Desktop/USP_Assignment$ ./prime numbers.txt
Child 0 completed the task: 7 is a prime number.
Child 1 completed the task: 20 is NOT a prime number.
Child 2 completed the task: 11 is a prime number.
Child 3 completed the task: 3849214 is NOT a prime number.
Child 4 completed the task: 19 is a prime number.
Child 6 completed the task: 99991 is a prime number.
Child 7 completed the task: 104729 is a prime number.
Child 8 completed the task: 1299709 is a prime number.
Child 9 completed the task: 15485863 is a prime number.
Child 10 completed the task: 32452843 is a prime number.
Child 11 completed the task: 49979687 is a prime number.
Child 5 completed the task: 67867967 is a prime number.
```

```
resham@resham-VirtualBox:~/Desktop/USP_Assignment$ ./prime numbers1.txt
Child 0 completed the task: 7 is a prime number.
Child 1 completed the task: 30 is NOT a prime number.
Child 2 completed the task: 15 is NOT a prime number.
Child 3 completed the task: 4869710 is NOT a prime number.
Child 4 completed the task: 18 is NOT a prime number.
Child 11 completed the task: 78 is NOT a prime number.
Child 6 completed the task: 729 is NOT a prime number.
Child 9 completed the task: 5409843 is NOT a prime number.
Child 10 completed the task: 493455687 is NOT a prime number.
Child 5 completed the task: 8761 is a prime number.
Child 8 completed the task: 178211863 is NOT a prime number.
Child 7 completed the task: 987659 is a prime number.
resham@resham-VirtualBox:~/Desktop/USP_Assignment$
```

```
resham@resham-VirtualBox:~/Desktop/USP_Assignment$ ./prime small_numbers.txt
Child 1 completed the task: 4 is NOT a prime number.
Child 0 completed the task: 2 is a prime number.
Child 2 completed the task: 5 is a prime number.
resham@resham-VirtualBox:~/Desktop/USP_Assignment$ ./prime single_number.txt
Child 0 completed the task: 99991 is a prime number.
resham@resham-VirtualBox:~/Desktop/USP_Assignment$ ./prime large_numbers.txt
Child 0 completed the task: 100 is NOT a prime number.
Child 2 completed the task: 102 is NOT a prime number.
Child 4 completed the task: 104 is NOT a prime number.
Child 1 completed the task: 101 is a prime number.
Child 3 completed the task: 103 is a prime number.
resham@resham-VirtualBox:~/Desktop/USP_Assignment$
```

2. Error Handling Test:

- **Extra Command Line Argument**: verified that the program displayed the proper usage message:

```
resham@resham-VirtualBox:~/Desktop/USP_Assignment$ ./prime numbers.txt extra
Usage: ./prime <filename>
resham@resham-VirtualBox:~/Desktop/USP_Assignment$
```

- **No File Given**: verified that the same usage message appeared and the program exited safely.

```
resham@resham-VirtualBox:~/Desktop/USP_Assignment$ ./prime
Usage: ./prime <filename>
resham@resham-VirtualBox:~/Desktop/USP_Assignment$
```

- **Non-existent File Name:** verified that the program printed the error message and terminated without crashing:

```
resham@resham-VirtualBox:~/Desktop/USP_Assignment$ ./prime nonexistent.txt
Error: cannot open file
resham@resham-VirtualBox:~/Desktop/USP_Assignment$
```

3. **Memory Management:**

   Valgrind was used to confirm there were no memory leaks or invalid memory accesses:

```
resham@resham-VirtualBox:~/Desktop/USP_Assignment$ valgrind --leak-check=full ./prime
==4007== Memcheck, a memory error detector
==4007== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==4007== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==4007== Command: ./prime
==4007==
Usage: ./prime <filename>
==4007==
==4007== HEAP SUMMARY:
==4007==     in use at exit: 0 bytes in 0 blocks
==4007==   total heap usage: 0 allocs, 0 frees, 0 bytes allocated
==4007==
==4007== All heap blocks were freed -- no leaks are possible
==4007==
==4007== For lists of detected and suppressed errors, rerun with: -s
==4007== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
resham@resham-VirtualBox:~/Desktop/USP_Assignment$
```

Conclusion

The program successfully implements Inter Process Communication using pipes and system calls. It correctly read integers, manages multiple child processes, and ensures clean termination and memory safety verifies through testing.